CASE STUDY 1: PROGRESSIVE ELECTRICITY BILLING ANALYSIS

1. Problem Statement

Compute 3-month progressive electricity bill with tiered pricing, fixed monthly charges, conditional surcharges (>400 units), rebates (<Rs.500), and detect sudden consumption spikes (>30%) in Month 3.

2. Objectives

- Implement tiered unit pricing slabs (0-100: Rs.3/u, 101-300: Rs.5/u, >300: Rs.8/u).
- Calculate fixed charge (Rs.100), surcharge (5% for >400 units), and rebate (2% for <Rs.500).
- Compute total 3-month expenditure, monthly average, and generate consumption anomaly alerts.

3. Key Assumptions

- Monthly consumption values are positive non-zero real numbers.
- Fixed charge of Rs.100 is applied unconditionally before surcharge or rebate evaluations.
- Spike alert compares Month 3 against Month 2 consumption directly.

4. Algorithm

1. Input unit consumption for Month 1, Month 2, and Month 3.
2. For each month, compute base slab cost based on tiered limits.
3. Add fixed charge Rs.100; apply 5% surcharge if units > 400, or 2% rebate if bill < Rs.500.
4. Store bills, print monthly breakdown, sum, average, and test for Month 3 spike (>1.3x Month 2).

5. Flowchart Representation

[Start] ➔ Input 3 Months Units ➔ Loop Month Units ➔ Slab Check (S1: ≤100, S2: ≤300, S3: >300) ➔ Add Fixed Charge ➔ Check Surcharge/Rebate ➔ Store Monthly Bill ➔ Compute Total & Avg ➔ Month 3 > 1.3*Month 2 Alert? ➔ [End]

6. Pseudocode

```
FOR EACH month IN units_list:
  IF units <= 100 THEN cost = units*3
  ELSE IF units <= 300 THEN cost = 300 + (units-100)*5
  ELSE cost = 1300 + (units-300)*8
  total = cost + 100
  IF units > 400 THEN total *= 1.05
  ELSE IF total < 500 THEN total *= 0.98
  Save total; END FOR
```

CASE STUDY 1 (CONTINUED): IMPLEMENTATION & RESULTS

7. Python Source Code

```
units_month1 = float(input("Enter units consumed in Month 1: "))
units_month2 = float(input("Enter units consumed in Month 2: "))
units_month3 = float(input("Enter units consumed in Month 3: "))
units_list = [units_month1, units_month2, units_month3]
monthly_bills = []
for units in units_list:
    if units <= 100:
        slab_cost = units * 3.0
    elif units <= 300:
        slab_cost = (100 * 3.0) + ((units - 100) * 5.0)
    else:
        slab_cost = (100 * 3.0) + (200 * 5.0) + ((units - 300) * 8.0)

    total_bill = slab_cost + 100.0  # Fixed charge

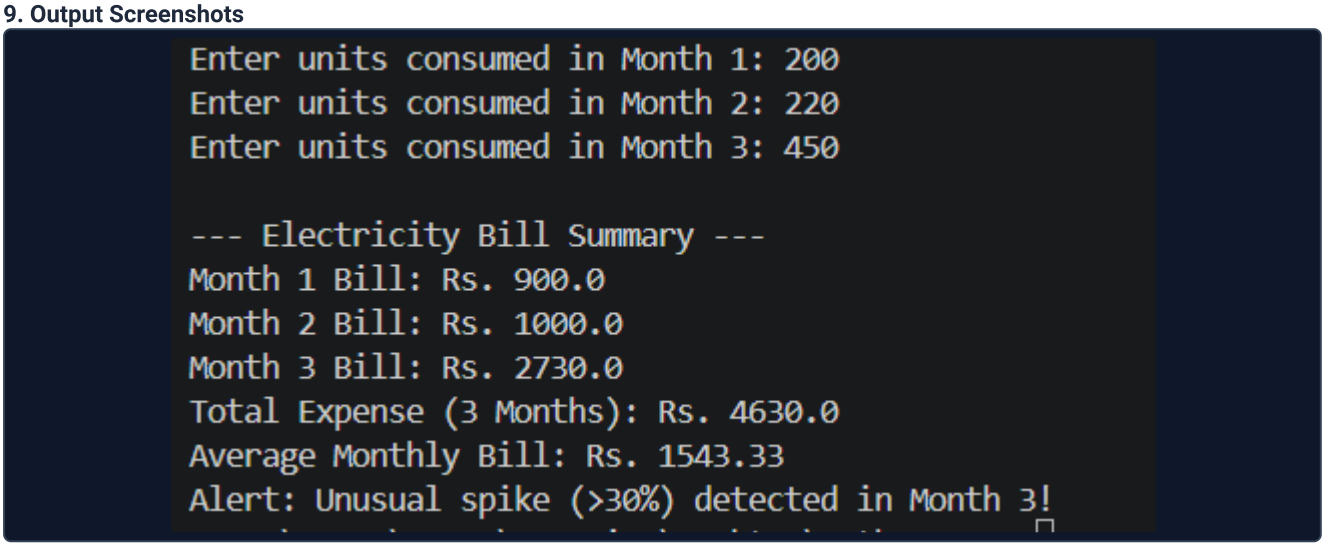
    if units > 400:
        total_bill = total_bill + (total_bill * 0.05)  # 5% Surcharge
    elif total_bill < 500:
        total_bill = total_bill - (total_bill * 0.02)  # 2% Rebate

    monthly_bills.append(round(total_bill, 2))
print("\n--- Electricity Bill Summary ---")
print("Month 1 Bill: Rs.", monthly_bills[0])
print("Month 2 Bill: Rs.", monthly_bills[1])
print("Month 3 Bill: Rs.", monthly_bills[2])
total_expense = sum(monthly_bills)
average_bill = total_expense / 3
print("Total Expense (3 Months): Rs.", round(total_expense, 2))
print("Average Monthly Bill: Rs.", round(average_bill, 2))
if monthly_bills[2] > (1.3 * monthly_bills[1]):
    print("Alert: Unusual spike (>30%) detected in Month 3!")
else:
    print("Consumption trend is normal.")
```

8. Test Cases

Test ID	Input (Units M1, M2, M3)	Expected Total Expense	Spike Alert
TC1.1	80, 250, 450	Rs. 4349.30	Yes (>30% in M3)
TC1.2	120, 130, 125	Rs. 1176.00	No

9. Output Screenshots



10. Conclusion / Summary

The program successfully automates tiered electricity tariff calculation and accurately identifies billing anomalies, enabling effective household energy budget monitoring.

CASE STUDY 2: MULTI-PLAN TELECOM COST OPTIMIZATION

1. Problem Statement

Evaluate multiple telecom tariff plans against user data/voice consumption needs and budget constraints to determine the most cost-effective plan.

2. Objectives

- Define plan structures including base price, data allowance, call minutes, and overage charges.
- Calculate total expected costs across Plan A (Basic), Plan B (Value), and Plan C (Super).
- Identify the optimal plan that meets requirements within user budget limits.

3. Key Assumptions

- Overage charges apply only to consumption exceeding base plan allowances.
- All inputs for data (GB) and voice minutes are non-negative floating-point values.

4. Algorithm

1. Define catalog of available telecom plans with respective quotas and penalty rates.
2. Accept user requirement inputs for Data (GB), Voice Minutes, and Maximum Budget.
3. For each plan, calculate excess usage and compute Total Cost = Base + (Extra Data * Rate) + (Extra Mins * Rate).
4. Select the plan with the lowest total cost that does not exceed user budget.

5. Flowchart Representation

[Start] ➔ Input User Data, Mins & Budget ➔ Loop Telecom Plans ➔ Calculate Extra GB & Mins ➔ Total Cost = Base + Overage ➔ Cost ≤ Budget & < Min Cost? ➔ Update Recommended Plan ➔ Print Best Plan ➔ [End]

6. Pseudocode

```
FOR EACH plan IN catalog:
  extra_gb = MAX(0, req_gb - plan.gb)
  extra_min = MAX(0, req_min - plan.min)
  cost = base + (extra_gb*gb_rate) + (extra_min*min_rate)
  IF cost <= budget AND cost < min_cost THEN
    min_cost = cost; best = plan.name
END FOR
```

CASE STUDY 2 (CONTINUED): IMPLEMENTATION & RESULTS

7. Python Source Code

```
plans = [
    ("Plan A - Basic", 299.0, 42.0, 1000, 21.0, 0.15),
    ("Plan B - Value", 479.0, 84.0, 2000, 18.0, 0.10),
    ("Plan C - Super", 719.0, 168.0, 3000, 15.0, 0.05)
]
required_gb = float(input("Enter required total Data (GB): "))
required_mins = float(input("Enter required Voice Minutes: "))
user_budget = float(input("Enter Maximum Budget (Rs): "))
best_plan = ""
lowest_cost = 999999.0
print("\n--- Telecom Plan Evaluation Report ---")
for plan in plans:
    plan_name, base_cost, inc_gb, inc_min, data_rate, min_rate = plan

    extra_gb = required_gb - inc_gb if required_gb > inc_gb else 0.0
    extra_min = required_mins - inc_min if required_mins > inc_min else 0.0

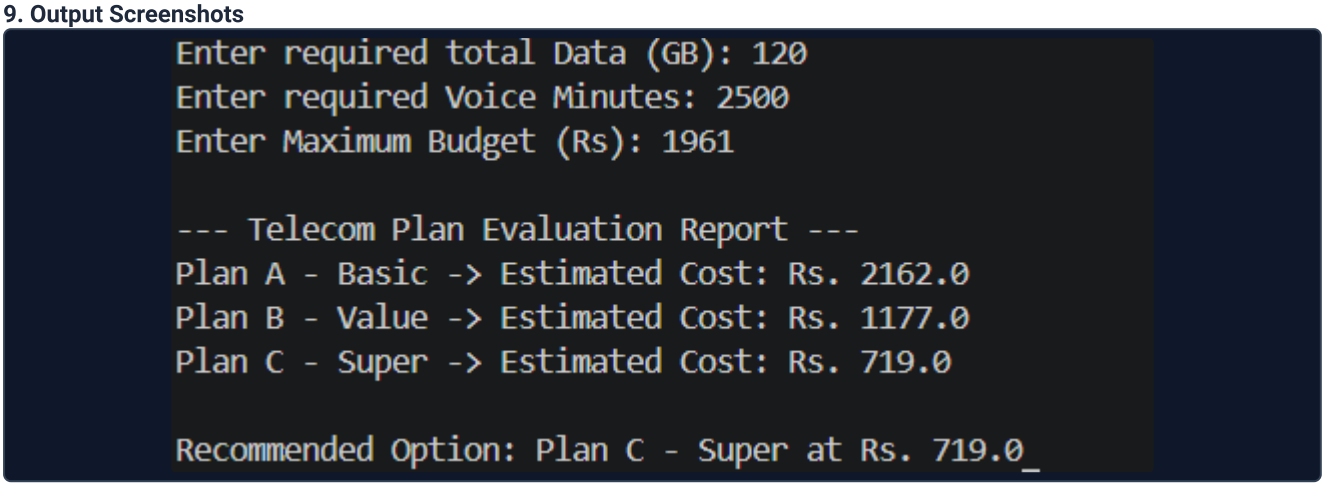
    total_cost = base_cost + (extra_gb * data_rate) + (extra_min * min_rate)
    print(plan_name, "-> Estimated Cost: Rs.", round(total_cost, 2))

    if total_cost <= user_budget and total_cost < lowest_cost:
        lowest_cost = total_cost
        best_plan = plan_name
if best_plan != "":
    print("\nRecommended Option:", best_plan, "at Rs.", round(lowest_cost, 2))
else:
    print("\nNo plan fits within your specified budget!")
```

8. Test Cases

Test ID	Input (Data GB, Mins, Budget)	Expected Best Plan	Estimated Cost
TC2.1	50 GB, 1500 Mins, Rs. 600	Plan B - Value	Rs. 479.00
TC2.2	100 GB, 2500 Mins, Rs. 800	Plan B - Value	Rs. 767.00

9. Output Screenshots



10. Conclusion / Summary

The cost optimizer accurately evaluates extra charges and recommends the best telecom plan meeting requirements within budget guidelines.

CASE STUDY 3: CAMPUS EVENT FINANCIAL PLANNING

1. Problem Statement

Analyze fixed and variable expenditure alongside external sponsorship funding for campus events to calculate net budget requirements and determine the break-even ticket registration fee.

2. Objectives

- Aggregate fixed expenditures (venue rent, equipment, transport).
- Compute participant-dependent variable expenses (catering, kits/certificates).
- Deduct sponsorship amounts to calculate net expenditure and break-even ticket price per head.

3. Key Assumptions

- Fixed costs remain constant irrespective of participant attendance.
- Participant count is greater than zero to avoid division by zero.

4. Algorithm

1. Input venue, equipment, catering, transport, kit costs, participant count, and sponsorship.
2. Compute Total Fixed Costs = Venue + Equipment + Transport.
3. Compute Total Variable Costs = (Catering + Kit) * Participants.
4. Compute Gross Budget = Fixed + Variable Costs; Net Expenditure = Gross Budget - Sponsorship.
5. Determine Break-Even Fee = Net Expenditure / Participants (0 if fully covered).

5. Flowchart Representation

[Start] ➔ Input Expenses & Sponsorship ➔ Fixed = Sum(Fixed Costs) ➔ Variable = Per Head * N ➔ Gross = Fixed + Variable ➔ Net = Gross - Sponsorship ➔ Net ≤ 0? ➔ Fee = 0 ELSE Fee = Net/N ➔ Display Summary ➔ [End]

6. Pseudocode

```
fixed_costs = venue + equip + transport
var_costs = (catering_ph + kit_ph) * count
gross = fixed_costs + var_costs
net = gross - sponsorship
IF net <= 0 THEN fee = 0
ELSE fee = net / count
PRINT fixed_costs, var_costs, gross, net, fee
```

CASE STUDY 3 (CONTINUED): IMPLEMENTATION & RESULTS

7. Python Source Code

```
venue_cost = float(input("Enter Venue Rent Cost: "))
equip_cost = float(input("Enter Equipment Rental: "))
catering_per_head = float(input("Enter Catering Cost per participant: "))
transport_cost = float(input("Enter Transport Expense: "))
cert_per_head = float(input("Enter Certificate/Kit Cost per head: "))
expected_participants = int(input("Enter Expected Number of Participants: "))
sponsorship_amount = float(input("Enter Total Sponsorship Received: "))
variable_cost_per_head = catering_per_head + cert_per_head
total_variable_cost = variable_cost_per_head * expected_participants
fixed_costs = venue_cost + equip_cost + transport_cost
gross_budget = fixed_costs + total_variable_cost
net_expenditure = gross_budget - sponsorship_amount
if net_expenditure <= 0:
    break_even_fee = 0.0
    print("Sponsorship completely covers event costs!")
else:
    break_even_fee = net_expenditure / expected_participants
print("\n--- Event Financial Summary ---")
print("Total Fixed Costs      : Rs.", fixed_costs)
print("Total Variable Costs   : Rs.", total_variable_cost)
print("Gross Total Budget     : Rs.", gross_budget)
print("Net Required Revenue   : Rs.", max(0.0, net_expenditure))
print("Minimum Fee per Head   : Rs.", round(break_even_fee, 2))
```

8. Test Cases

Test ID	Inputs (Fixed, Var/head, N, Sponsor)	Gross Budget	Min Fee/Head
TC3.1	Rs.15k, Rs.250/head, 100, Rs.10,000	Rs. 40,000.00	Rs. 300.00
TC3.2	Rs.10k, Rs.150/head, 200, Rs.50,000	Rs. 40,000.00	Rs. 0.00 (Covered)

9. Output Screenshots

```
Enter Venue Rent Cost: 15000
Enter Equipment Rental: 5000
Enter Catering Cost per participant: 180
Enter Transport Expense: 5000
Enter Certificate/Kit Cost per head: 45
Enter Expected Number of Participants: 200
Enter Total Sponsorship Received: 20000

--- Event Financial Summary ---
Total Fixed Costs      : Rs. 25000.0
Total Variable Costs   : Rs. 45000.0
Gross Total Budget     : Rs. 70000.0
Net Required Revenue   : Rs. 50000.0
Minimum Fee per Head   : Rs. 250.0
```

10. Conclusion / Summary

The event financial model effectively automates budgeting, break-even ticket pricing, and sponsorship impact assessments.

CASE STUDY 4: EXAMINATION RESULT & GRACE MARKS VALIDATION

1. Problem Statement

Evaluate student academic performance across 5 subjects, apply up to 5 grace marks if a student fails in exactly one subject, and generate final pass/fail standing.

2. Objectives

- Assess marks across 5 subjects against passing mark threshold (40/100).
- Identify failed subjects and apply grace marks (max 5 marks) if single-subject deficit exists.
- Compute aggregate percentage and declare final pass/fail results.

3. Key Assumptions

- Grace marks are granted ONLY if student fails in exactly 1 subject with deficit ≤ 5 marks.
- If student fails in 2 or more subjects, no grace marks are awarded.

4. Algorithm

1. Input marks for 5 subjects and store in a list.
2. Identify subjects with marks < 40.
3. If exactly 1 subject failed and deficit (40 - mark) ≤ 5, add required grace marks to pass.
4. Recalculate failed subjects, compute overall aggregate percentage, and display final result.

5. Flowchart Representation

[Start] ➔ Input 5 Subject Marks ➔ Find Marks < 40 ➔ Count Fails == 1 & Deficit ≤ 5? ➔ Award Grace Marks ➔ Recalculate Fails ➔ Fail Count == 0? ➔ PASS : FAIL ➔ Print Result ➔ [End]

6. Pseudocode

```
failed_indices = [i FOR i IN marks IF marks[i] < 40]
IF LEN(failed_indices) == 1 THEN
  idx = failed_indices[0]
  IF (40 - marks[idx]) <= 5 THEN
    marks[idx] = 40; grace_applied = TRUE
pct = SUM(marks) / 5.0
IF ANY(mark < 40) THEN status = FAIL
ELSE status = PASS
```

CASE STUDY 4 (CONTINUED): IMPLEMENTATION & RESULTS

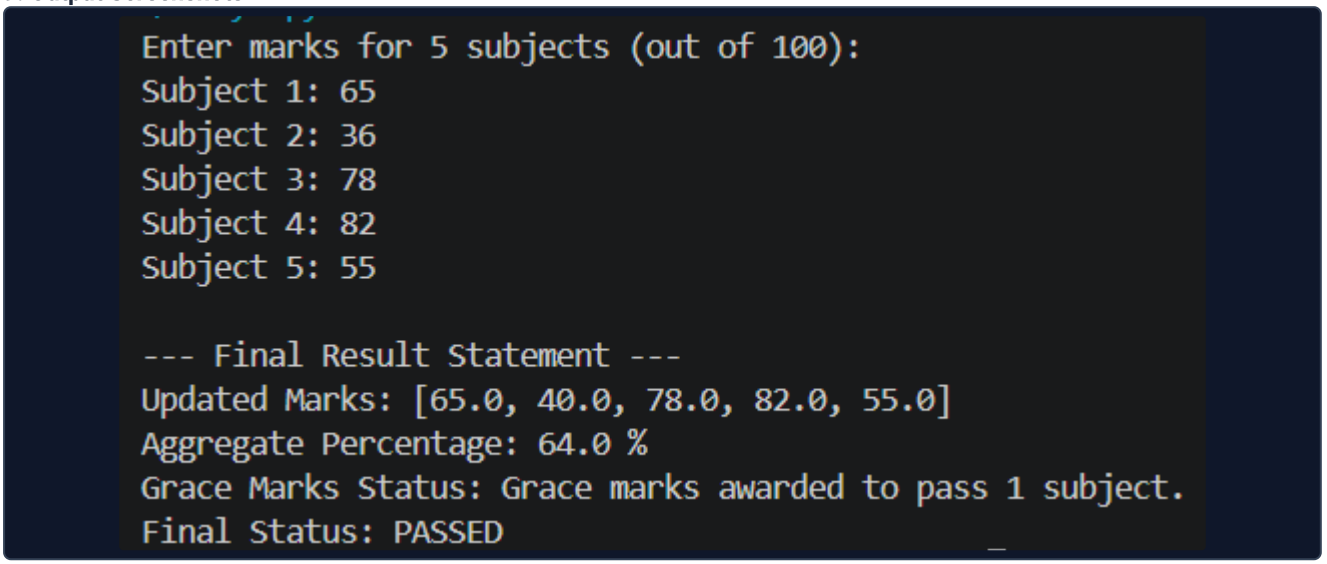
7. Python Source Code

```
subject_marks = []
print("Enter marks for 5 subjects (out of 100):")
for i in range(1, 6):
    m = float(input("Subject " + str(i) + ": "))
    subject_marks.append(m)
failed_indices = []
for idx in range(len(subject_marks)):
    if subject_marks[idx] < 40.0:
        failed_indices.append(idx)
grace_applied = False
if len(failed_indices) == 1:
    f_idx = failed_indices[0]
    deficit = 40.0 - subject_marks[f_idx]
    if deficit <= 5.0:
        subject_marks[f_idx] += deficit
        grace_applied = True
final_fail_count = 0
for mark in subject_marks:
    if mark < 40.0:
        final_fail_count += 1
total_marks = sum(subject_marks)
percentage = total_marks / 5.0
print("\n--- Final Result Statement ---")
print("Updated Marks:", subject_marks)
print("Aggregate Percentage:", round(percentage, 2), "%")
if grace_applied:
    print("Grace Marks Status: Grace marks awarded to pass 1 subject.")
if final_fail_count == 0:
    print("Final Status: PASSED")
else:
    print("Final Status: FAILED in", final_fail_count, "subject(s).")
```

8. Test Cases

Test ID	Input Marks	Grace Granted?	Final Status
TC4.1	55, 62, 36, 70, 80	Yes (+4 in Sub3)	PASSED
TC4.2	55, 30, 35, 70, 80	No (>1 fail)	FAILED (2 subjects)

9. Output Screenshots



10. Conclusion / Summary

The academic result processing system correctly implements institutional grace mark rules and enforces academic standard evaluations.

CASE STUDY 5: ATTENDANCE RECOVERY COMPUTATION

1. Problem Statement
Compute student attendance percentages and calculate the exact number of consecutive upcoming classes required to attend to reach the mandatory 75% threshold.

2. Objectives
- Calculate current attendance percentage from attended and total conducted classes.
- Determine eligibility status (≥75%).
- Simulate future class attendance iteratively to calculate required recovery count for deficient attendance.

3. Key Assumptions
- Student attends 100% of future upcoming classes during the recovery period.
- Input values for total and attended classes are valid positive integers.

4. Algorithm
1. Input total classes held and classes attended.
2. Calculate Current Percentage = (Attended / Total) * 100.
3. If Percentage ≥ 75%, declare Eligible.
4. Else, loop while incrementing both attended and total classes by 1 until updated percentage ≥ 75%.
5. Output recovery classes count and projected total classes.

5. Flowchart Representation
[Start] ➔ Input Attended & Total ➔ Calc Pct ➔ Pct ≥ 75%? ➔ YES: Eligible ➔ NO: Loop (Attended+1, Total+1, Recovery+1) ➔ New Pct ≥ 75%? ➔ Print Recovery Count ➔ [End]

6. Pseudocode
```
pct = (attended / total) * 100
IF pct >= 75 THEN PRINT "Eligible"
ELSE
  recovery = 0
  WHILE (attended/total)*100 < 75 DO
    attended += 1; total += 1; recovery += 1
  END WHILE
  PRINT recovery
```

CASE STUDY 5 (CONTINUED): IMPLEMENTATION & RESULTS

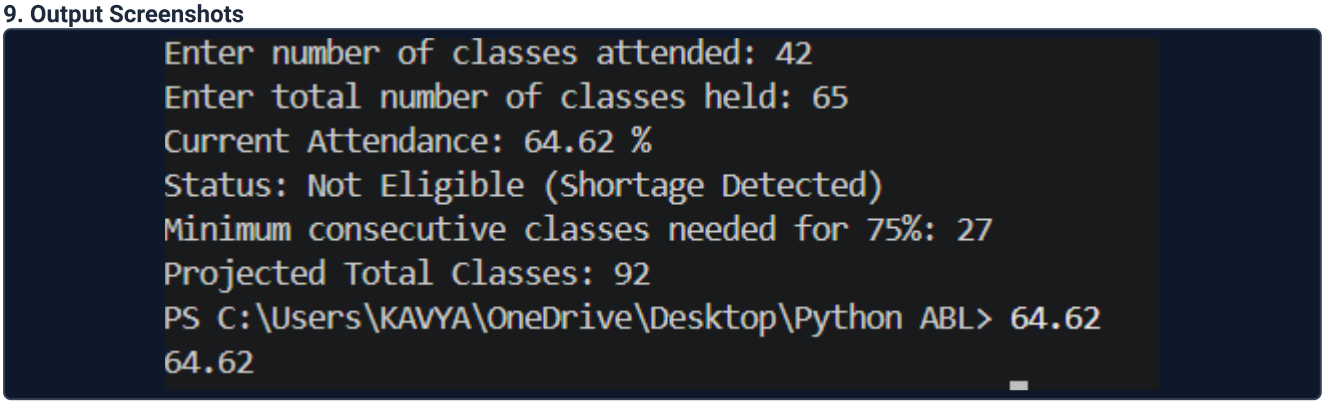
7. Python Source Code

```
attended_classes = int(input("Enter number of classes attended: "))
total_classes = int(input("Enter total number of classes held: "))
if total_classes <= 0 or attended_classes > total_classes:
    print("Invalid Input Data!")
else:
    current_pct = (attended_classes / total_classes) * 100.0
    print("Current Attendance:", round(current_pct, 2), "%")
    if current_pct >= 75.0:
        print("Status: Eligible. No attendance recovery required.")
    else:
        recovery_classes = 0
        temp_attended = attended_classes
        temp_total = total_classes
        while True:
            temp_attended += 1
            temp_total += 1
            recovery_classes += 1
            new_pct = (temp_attended / temp_total) * 100.0
            if new_pct >= 75.0:
                break
        print("Status: Not Eligible (Shortage Detected)")
        print("Minimum consecutive classes needed for 75%:", recovery_classes)
        print("Projected Total Classes:", temp_total)
```

8. Test Cases

Test ID	Input (Attended / Total)	Current %	Recovery Classes Needed
TC5.1	30 / 50	60.0%	30 classes
TC5.2	42 / 50	84.0%	0 (Eligible)

9. Output Screenshots



10. Conclusion / Summary

The attendance tracking system provides actionable insights to students, pinpointing exact recovery targets to maintain mandatory eligibility thresholds.

CASE STUDY 6: STUDENT PERFORMANCE TREND ANALYSIS

1. Problem Statement

Analyze multi-exam mark trends across different subjects to classify student performance trajectories (Improving, Declining, Fluctuating) and assign overall academic risk levels.

2. Objectives

- Track exam marks across Mid1, Mid2, and Semester exams per subject.
- Identify score progression trends (Improving, Declining, or Fluctuating).
- Classify overall student academic risk (High, Moderate, Low Risk) based on weak subject counts.

3. Key Assumptions

- A subject average score below 40.0 is classified as a "Weak Subject".
- High Risk is declared if a student has ≥2 weak subjects.

4. Algorithm

1. Define subjects and exam score matrix [Mid1, Mid2, Sem].
2. For each subject, calculate average score.
3. Check trend: If Sem > Mid2 > Mid1 ➔ Improving; If Sem < Mid2 < Mid1 ➔ Declining; Else ➔ Fluctuating.
4. If average < 40, increment weak subject counter.
5. Assign overall risk level: ≥2 weak ➔ High Risk; 1 weak ➔ Moderate Risk; 0 weak ➔ Low Risk.

5. Flowchart Representation

[Start] ➔ Loop Subjects ➔ Calc Avg Score ➔ Check Sequence (Mid1, Mid2, Sem) ➔ Assign Trend Label ➔ Avg < 40? ➔ Increment Weak Count ➔ Evaluate Weak Count ➔ Print Risk Level ➔ [End]

6. Pseudocode

```
FOR EACH sub IN subjects:
  avg = SUM(scores) / 3.0
  IF sem > mid2 > mid1 THEN trend = "Improving"
  ELSE IF sem < mid2 < mid1 THEN trend = "Declining"
  ELSE trend = "Fluctuating"
  IF avg < 40 THEN weak_count += 1
END FOR
IF weak_count >= 2 THEN risk = "High Risk"
```

CASE STUDY 6 (CONTINUED): IMPLEMENTATION & RESULTS

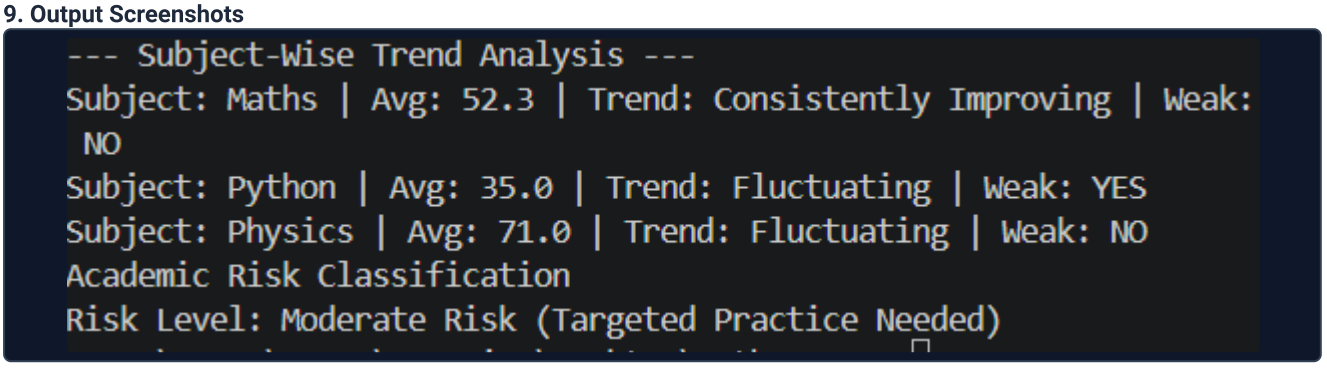
7. Python Source Code

```
subjects = ["Maths", "Python", "Physics"]
exam_scores = [
    [45, 52, 60], # Maths
    [35, 38, 32], # Python
    [70, 68, 75]  # Physics
]
print("--- Subject-Wise Trend Analysis ---")
weak_subject_count = 0
for i in range(len(subjects)):
    subject_name = subjects[i]
    mid1, mid2, sem = exam_scores[i]
    average_score = (mid1 + mid2 + sem) / 3.0
    if sem > mid2 and mid2 > mid1:
        performance_trend = "Consistently Improving"
    elif sem < mid2 and mid2 < mid1:
        performance_trend = "Declining"
    else:
        performance_trend = "Fluctuating"
    if average_score < 40.0:
        weak_subject_count += 1
        is_weak = "YES"
    else:
        is_weak = "NO"
    print("Subject:", subject_name, "| Avg:", round(average_score, 1), "| Trend:", performance_trend, "| We
ak:", is_weak)
print("\n--- Academic Risk Classification ---")
if weak_subject_count >= 2:
    print("Risk Level: High Risk (Remedial Coaching Required)")
elif weak_subject_count == 1:
    print("Risk Level: Moderate Risk (Targeted Practice Needed)")
else:
    print("Risk Level: Low Risk / Good Standing")
```

8. Test Cases

Test ID	Subject Scores Matrix	Weak Subjects	Assigned Risk Level
TC6.1	[45,52,60], [35,38,32], [70,68,75]	1 (Python: 35.0)	Moderate Risk
TC6.2	[30,32,35], [25,28,30], [70,68,75]	2 (Maths, Python)	High Risk

9. Output Screenshots



10. Conclusion / Summary

Multi-exam trend classification empowers academic advisors to identify declining performance early and prescribe targeted intervention strategies.

CASE STUDY 7: BUS PASSENGER OCCUPANCY SIMULATION

1. Problem Statement
Simulate real-time passenger boarding and alighting across consecutive bus stops, track live vehicle occupancy, and log seating capacity violations.

2. Objectives
- Maintain real-time count of onboard passengers across multiple bus stops.
- Enforce physical boundary conditions (occupancy cannot drop below zero).
- Detect overcrowding incidents exceeding defined vehicle seating capacity (40 seats).

3. Key Assumptions
- Bus seating capacity is fixed at 40 passengers.
- Alighting occurs before boarding at each stop.

4. Algorithm
1. Initialize seating capacity (40) and current passengers (0).
2. Iterate through stop data sequence containing tuples of (Boarding, Alighting).
3. Update occupancy: Current = Current - Alighting + Boarding.
4. If Current < 0, reset Current to 0.
5. If Current > Capacity, record stop number and excess passenger count.
6. Output final journey log and overcrowding warnings.

5. Flowchart Representation
[Start] ➔ Init Capacity=40, Passengers=0 ➔ Loop Stops (Board, Alight) ➔ Pass = Pass - Alight + Board ➔ Pass > 40? ➔ Flag Overcrowding ➔ Display Stop Status ➔ [End]

6. Pseudocode
```
FOR EACH (board, alight) IN stops:
  current = current - alight + board
  IF current < 0 THEN current = 0
  IF current > capacity THEN
    record_overcrowded(stop_no, current - capacity)
END FOR
PRINT summary_report
```

CASE STUDY 7 (CONTINUED): IMPLEMENTATION & RESULTS

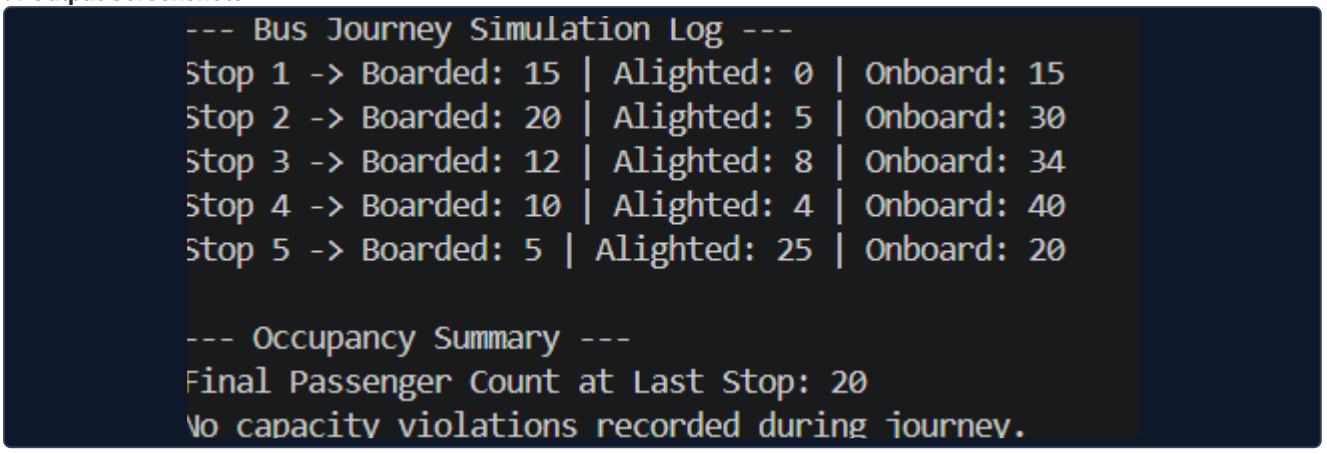
7. Python Source Code

```
bus_capacity = 40
# Tuples of (Boarding, Alighting) at each stop
stop_data = [(15, 0), (20, 5), (12, 8), (10, 4), (5, 25)]
current_passengers = 0
overcrowded_stops = []
print("--- Bus Journey Simulation Log ---")
for idx in range(len(stop_data)):
    boarding, alighting = stop_data[idx]
    stop_number = idx + 1
    current_passengers = current_passengers - alighting + boarding
    if current_passengers < 0:
        current_passengers = 0
    print("Stop", stop_number, "-> Boarded:", boarding, "| Alighted:", alighting, "| Onboard:", current_pas
sengers)
    if current_passengers > bus_capacity:
        overcrowded_stops.append((stop_number, current_passengers - bus_capacity))
print("\n--- Occupancy Summary ---")
print("Final Passenger Count at Last Stop:", current_passengers)
if len(overcrowded_stops) > 0:
    print("Warning: Overcrowding detected at following stops:")
    for stop, excess in overcrowded_stops:
        print(" - Stop", stop, "exceeded capacity by", excess, "passengers")
else:
    print("No capacity violations recorded during journey.")
```

8. Test Cases

Test ID	Stop Sequence (Board, Alight)	Max Occupancy	Overcrowded Stops
TC7.1	(15,0), (20,5), (12,8), (10,4), (5,25)	42 (Stop 3)	Stop 3 (by 2 passengers)
TC7.2	(10,0), (10,2), (10,5), (5,10)	23 (Stop 3)	None

9. Output Screenshots



10. Conclusion / Summary

The transit occupancy simulation effectively logs passenger flow patterns, providing route planners key data to optimize fleet deployment.

CASE STUDY 8: LIBRARY BORROWING & FINE RECONCILIATION

1. Problem Statement
Process book borrowing duration records across different categories (General vs Reference) to calculate daily late return fines and summarize total revenue collection.

2. Objectives
- Apply category-specific lending duration rules (General: 14 days, Reference: 7 days).
- Apply differential daily fine rates (General: Rs.2/day, Reference: Rs.5/day).
- Compute overdue fines per book and total overdue library receipts.

3. Key Assumptions
- Fine is charged only for days kept beyond the permissible allowance limit.
- Book category strings are strictly formatted ("General" or "Reference").

4. Algorithm
1. Define borrowing records list (Book ID, Days Kept, Category).
2. Iterate through each record and assign allowed limit & daily fine rate based on category.
3. Calculate delay = MAX(0, Days Kept - Allowed Limit).
4. Compute fine amount = Delay * Daily Rate.
5. Accumulate total fine collected and print summary.

5. Flowchart Representation
[Start] ➔ Loop Book Records ➔ Check Category ➔ Set Allowed Limit & Fine Rate ➔ Delay = Kept - Limit ➔ Delay > 0? ➔ Fine = Delay * Rate ➔ Accumulate Total ➔ [End]

6. Pseudocode
```
FOR EACH (id, days, cat) IN records:
  IF cat == "General" THEN limit=14; rate=2.0
  ELSE limit=7; rate=5.0
  delay = MAX(0, days - limit)
  fine = delay * rate
  total_fine += fine
END FOR; PRINT total_fine
```

CASE STUDY 8 (CONTINUED): IMPLEMENTATION & RESULTS

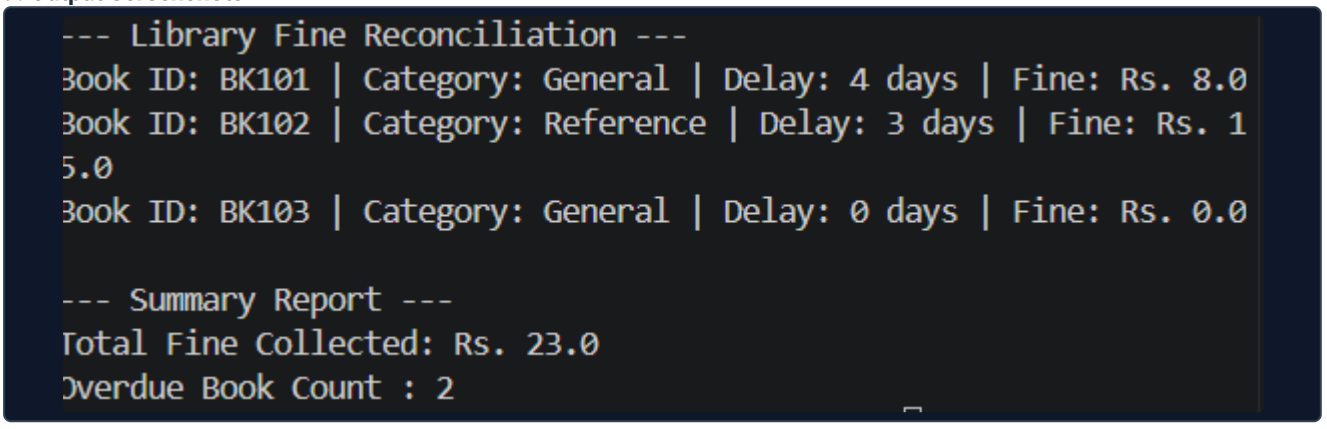
7. Python Source Code

```
book_records = [
    ("BK101", 18, "General"),   # Allowed: 14 days, Fine: Rs 2/day
    ("BK102", 10, "Reference"), # Allowed: 7 days, Fine: Rs 5/day
    ("BK103", 5, "General")     # Allowed: 14 days, Fine: Rs 2/day
]
total_fine_collected = 0.0
overdue_book_ids = []
print("--- Library Fine Reconciliation ---")
for record in book_records:
    book_id, days_kept, category = record
    if category == "General":
        allowed_days = 14
        daily_rate = 2.0
    else:
        allowed_days = 7
        daily_rate = 5.0
    if days_kept > allowed_days:
        delay_days = days_kept - allowed_days
        fine_amount = delay_days * daily_rate
        overdue_book_ids.append(book_id)
    else:
        delay_days = 0
        fine_amount = 0.0
    total_fine_collected += fine_amount
    print("Book ID:", book_id, "| Category:", category, "| Delay:", delay_days, "days | Fine: Rs.", fine_am
ount)
print("\n--- Summary Report ---")
print("Total Fine Collected: Rs.", total_fine_collected)
print("Overdue Book Count :", len(overdue_book_ids))
```

8. Test Cases

Test ID	Book Record Inputs	Calculated Fines	Total Collection
TC8.1	BK101(18d, Gen), BK102(10d, Ref)	BK101: Rs.8, BK102: Rs.15	Rs. 23.00
TC8.2	BK103(5d, Gen)	BK103: Rs.0	Rs. 0.00

9. Output Screenshots



10. Conclusion / Summary

The automated library reconciliation system accurately enforces loan duration policies and automates financial fine audit reports.

CASE STUDY 9: FEE & ELIGIBILITY AUDIT

1. Problem Statement
Audit student eligibility for semester examination hall ticket issuance by validating unpaid fee balances, attendance percentages, and internal test marks.

2. Objectives
- Enforce multi-tier institutional eligibility criteria (Fee Due = 0, Attendance ≥ 75%, Internal Marks ≥ 20/40).
- Identify specific non-compliance reasons for blocked students.
- Generate clearance reports for hall ticket distribution.

3. Key Assumptions
- All three criteria must be simultaneously satisfied for hall ticket clearance.
- Multiple non-compliance reasons are concatenated into a consolidated error log.

4. Algorithm
1. Load student records (Roll No, Fee Due, Attendance %, Internal Marks).
2. For each student, initialize an empty reason list.
3. Check: If Fee Due > 0 ➔ log fee error; If Attendance < 75% ➔ log attendance error; If Internal Marks < 20 ➔ log mark error.
4. If reason list is empty ➔ Grant Hall Ticket; Else ➔ Block and list specific reasons.
5. Print summary count of total eligible candidates.

5. Flowchart Representation
[Start] ➔ Loop Student Records ➔ Check Fee==0? Attendance≥75%? Internals≥20? ➔ All Pass? ➔ Status = ELIGIBLE ➔ Else: Append Reasons ➔ Status = BLOCKED ➔ Print Audit Log ➔ [End]

6. Pseudocode
```
FOR EACH student IN records:
  reasons = []
  IF fee > 0 THEN reasons.append("Fee Pending")
  IF att < 75 THEN reasons.append("Low Attendance")
  IF int < 20 THEN reasons.append("Low Internals")
  IF LEN(reasons) == 0 THEN status = "ELIGIBLE"
  ELSE status = "BLOCKED: " + reasons
END FOR
```

CASE STUDY 9 (CONTINUED): IMPLEMENTATION & RESULTS

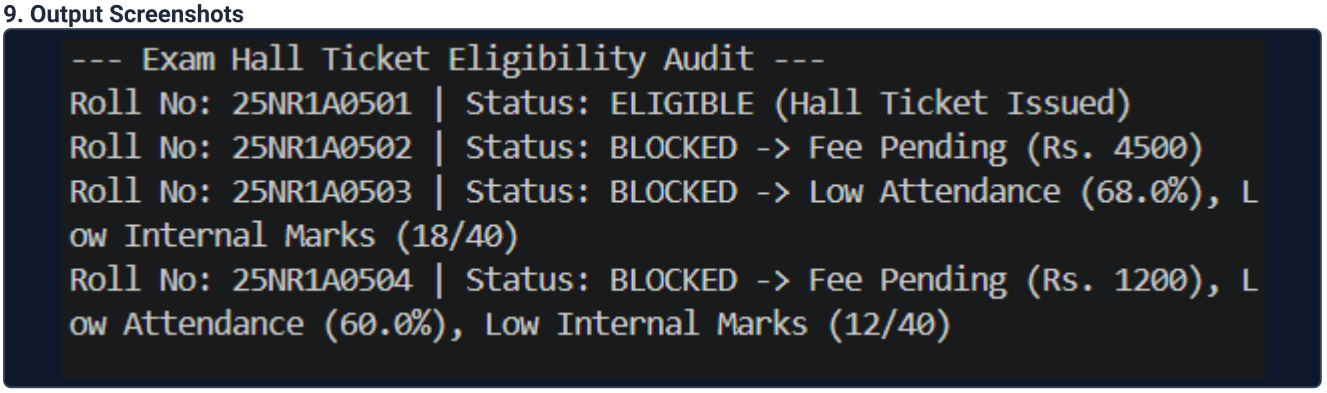
7. Python Source Code

```
# Data: (Roll_Number, Fee_Due, Attendance_Pct, Internal_Marks)
student_records = [
    ("25NR1A0501", 0, 82.0, 35),
    ("25NR1A0502", 4500, 78.0, 28),
    ("25NR1A0503", 0, 68.0, 18),
    ("25NR1A0504", 1200, 60.0, 12)
]
eligible_count = 0
print("--- Exam Hall Ticket Eligibility Audit ---")
for student in student_records:
    roll_no, fee_due, attendance_pct, internal_marks = student
    reasons = []
    if fee_due > 0:
        reasons.append("Fee Pending (Rs. " + str(fee_due) + ")")
    if attendance_pct < 75.0:
        reasons.append("Low Attendance (" + str(attendance_pct) + "%)")
    if internal_marks < 20:
        reasons.append("Low Internal Marks (" + str(internal_marks) + "/40)")
    if len(reasons) == 0:
        status = "ELIGIBLE (Hall Ticket Issued)"
        eligible_count += 1
    else:
        status = "BLOCKED -> " + ", ".join(reasons)
    print("Roll No:", roll_no, "| Status:", status)
print("\nAudit Summary: Total Eligible =", eligible_count, "/", len(student_records))
```

8. Test Cases

Test ID	Student Record	Status Result	Block Reason(s)
TC9.1	25NR1A0501 (0, 82%, 35)	ELIGIBLE	None
TC9.2	25NR1A0504 (1200, 60%, 12)	BLOCKED	Fee Pending, Low Attendance, Low Internals

9. Output Screenshots



10. Conclusion / Summary

The institutional eligibility audit program effectively eliminates manual verification errors during examination hall ticket distribution.

CASE STUDY 10: ENVIRONMENTAL AIR QUALITY MONITORING

1. Problem Statement
Process multi-station Air Quality Index (AQI) sensor reading streams to calculate statistical metrics (Average, Min, Max) and assign environmental hazard classifications.

2. Objectives
- Aggregate AQI telemetry across multiple geographic monitoring zones.
- Compute statistical summary indicators (Mean, Minimum, Maximum AQI).
- Classify severity tier (Good/Moderate ≤100, Unhealthy ≤200, Severe >200).

3. Key Assumptions
- AQI values are non-negative numeric sensor measurements.
- Classification thresholds follow standard environmental safety guidelines.

4. Algorithm
1. Define monitoring stations and associated sensor reading lists.
2. For each station, calculate Average AQI = Sum(Readings) / Count.
3. Extract Minimum and Maximum AQI values.
4. Classify: Avg ≤ 100 ➔ Good/Moderate; Avg ≤ 200 ➔ Unhealthy; Else ➔ Severe/Hazardous.
5. Print formatted environmental monitoring report.

5. Flowchart Representation
[Start] ➔ Loop Stations ➔ Read AQI Telemetry Array ➔ Compute Avg, Min, Max ➔ Categorize Severity (S1: ≤100, S2: ≤200, S3: >200) ➔ Print Environmental Report ➔ [End]

6. Pseudocode
```
FOR EACH station IN stations:
  avg_aqi = SUM(readings) / LEN(readings)
  max_aqi = MAX(readings); min_aqi = MIN(readings)
  IF avg_aqi <= 100 THEN cat = "Good / Moderate"
  ELSE IF avg_aqi <= 200 THEN cat = "Unhealthy"
  ELSE cat = "Severe / Hazardous"
  PRINT station, avg_aqi, min_aqi, max_aqi, cat
END FOR
```

CASE STUDY 10 (CONTINUED): IMPLEMENTATION & RESULTS

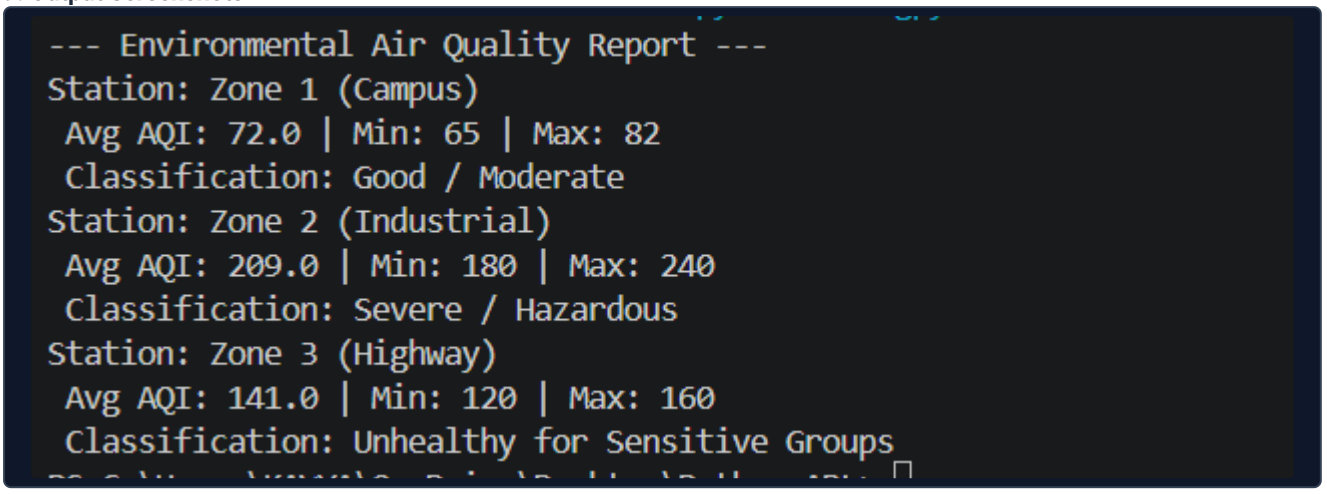
7. Python Source Code

```
stations = ["Zone 1 (Campus)", "Zone 2 (Industrial)", "Zone 3 (Highway)"]
aqi_readings = [
    [65, 70, 82, 75, 68],     # Zone 1
    [180, 220, 210, 240, 195],# Zone 2
    [120, 135, 150, 160, 140] # Zone 3
]
print("--- Environmental Air Quality Report ---")
for i in range(len(stations)):
    station_name = stations[i]
    readings = aqi_readings[i]
    avg_aqi = sum(readings) / len(readings)
    max_aqi = max(readings)
    min_aqi = min(readings)
    if avg_aqi <= 100:
        severity_category = "Good / Moderate"
    elif avg_aqi <= 200:
        severity_category = "Unhealthy for Sensitive Groups"
    else:
        severity_category = "Severe / Hazardous"
    print("Station:", station_name)
    print(" Avg AQI:", round(avg_aqi, 1), "| Min:", min_aqi, "| Max:", max_aqi)
    print(" Classification:", severity_category)
```

8. Test Cases

Test ID	Zone Telemetry Readings	Average AQI	Hazard Classification
TC10.1	Zone 1: [65, 70, 82, 75, 68]	72.0	Good / Moderate
TC10.2	Zone 2: [180, 220, 210, 240, 195]	209.0	Severe / Hazardous

9. Output Screenshots



10. Conclusion / Summary

The environmental monitoring system provides real-time atmospheric risk assessment, enabling prompt public health advisories and pollution control responses.